

Lecture 06

RISC-V: Machine Language and Microarchitecture

Instructor: Dr. Muhammad Imran



Computer Architecture | RISC-V Assembly Language

Acknowledgement

- Content from following has been used in these lectures
 - Computer Organization and Design, RISC-V 2nd Edition, Patterson and Hennessy
 - CS 61C: Great Ideas in Computer Architecture (Machine Structures), UC Berkeley

Contents

- Machine Language
- Instructions Encoding and Microarchitecture
- Arithmetic, Logical and Shift
- Data Transfer & Upper Immediate
- Program Control Instructions
- Summary of Instruction Formats

Machine Language



Encoding Instructions in Machine Language

- Assembly languages are primarily useful because they can be directly translated into binary code that can be run by a CPU.
- RISC-V has a particularly simple structure: Each instruction is translated into instructions of the same length; for RV32 (the version we learn in this class), each instruction is 32 bits (4 bytes) long.

Encoding Instructions in Machine Language

- Different instructions require different values
 - "add" specifies 3 register inputs
 - "addi" specifies 2 registers and 1 immediate
- As such, we define multiple formats, with each instruction getting encoded in its format.

Encoding Instructions in Machine Language

- Overall design philosophy: Split the 32 bits into "chunks" for each component of an instruction, and try to overlap these chunks as much as possible to simplify the underlying circuit.
- Most of this information is presented in compressed form on our reference card, so there's no need to memorize the exact numbers (such as opcode) associated with each instruction!

Instructions Encoding and Microarchitecture



Classes of Instructions

- **Arithmetic**

- Example
 - `add x5, x6, x7`
 - `sub x7, x9, x10`

- **Logical and Shift**

- Examples
 - `xor x5, x6, x7`
 - `srl x5, x6, x7`

- **Data Transfer**

- Example
 - `lw x5, 40(x6)` # load word
 - `sw x7, 0(x2)` # store word

- **Program Control / Branch**

- Example
 - `beq x1, x2, loop`
 - `jal x1, target`

Arithmetic Instructions – Machine Language

- R-type Encoding Format
 - When all operands are registers

Type	Field						Operations
(field size)	7 bits	5 bits	5 bits	3 bits	5 bits	7 bits	Arithmetic, Logical and Shift
R-type	func7	rs2	rs1	func3	rd	opcode	

- Example
 - add x9, x20, x21

31						0	
func7	rs2	rs1	func3	rd	opcode		
7 bits	5 bits	5 bits	3 bits	5 bits	7 bits		
0	21	20	0	9	51		Decimal
0000000	10101	10100	000	01001	0110011		Binary
0000 0001 0101 1010 0000 0100 1011 0011							
015A04B3							Hex

Arithmetic Instructions – Machine Language

- I-type Encoding Format
 - When one source operand is immediate constant value

Type		Field					Operations
(field size)	7 bits	5 bits	5 bits	3 bits	5 bits	7 bits	Arithmetic, Logical, Shift and Loads!
I-type	immediate[11:0]		rs1	func3	rd	opcode	

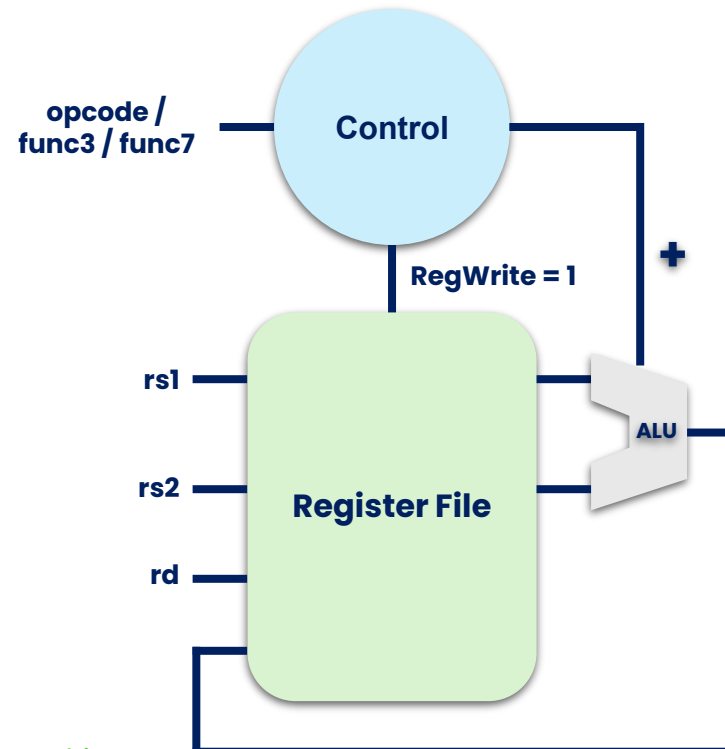
- Immediate value is always represented in 2's complement notation!
- Example
 - addi x1, x2, -1

31					0
immediate[11:0]		rs1	funct3	rd	opcode
12 bits		5 bits	3 bits	5 bits	7 bits
-1		2	0	1	19
1111111111		00010	000	00001	0010011
1111 1111 1111 0001 0000 0000 1001 0011					
FFF10093					

Arithmetic Instructions – Microarchitecture

- How arithmetic instructions can be implemented?
 - Example – R-type
 - add x1, x2, x3

R-type	func7	rs2	rs1	func3	rd	opcode
--------	-------	-----	-----	-------	----	--------



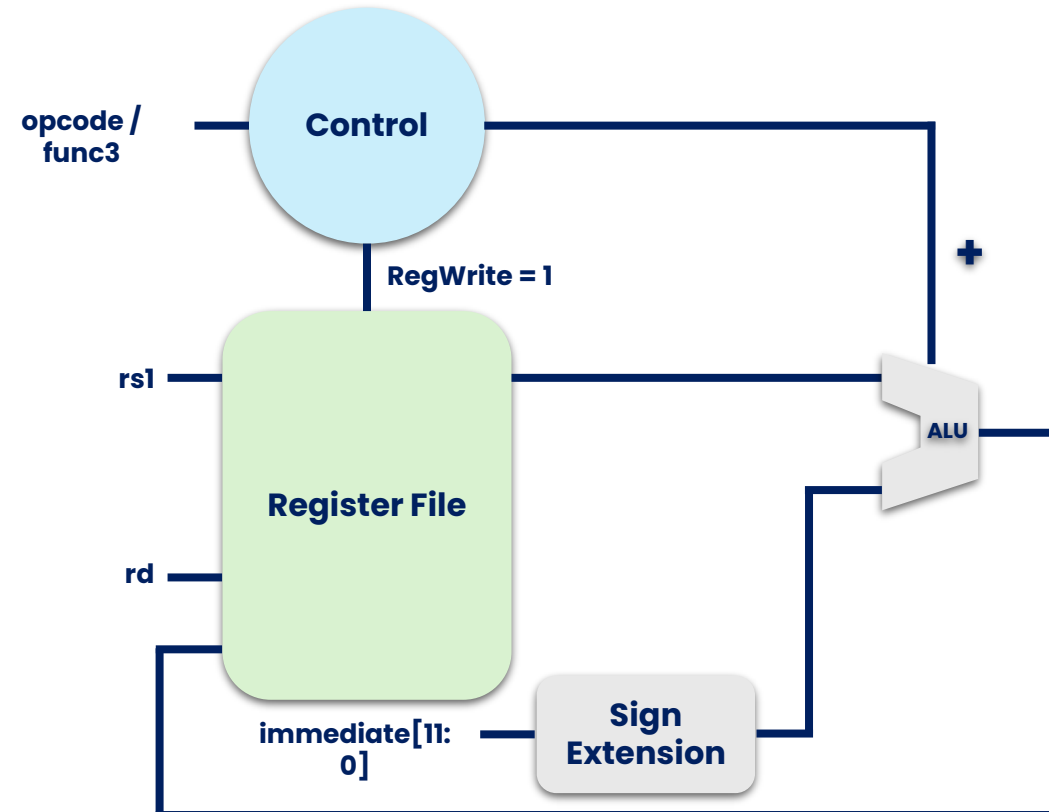
Arithmetic Instructions – Microarchitecture

- How arithmetic instructions can be implemented?
 - Example – I-type
 - add x1, x2, -1

I-type	immediate[11:0]	rs1	func3	rd	opcode
--------	-----------------	-----	-------	----	--------

Sign Extension
12-bit value to 32-bit value
(preserve signed number)

Example
1111 = -1
11111111 = -1



Logical and Shift Instructions – Machine Language

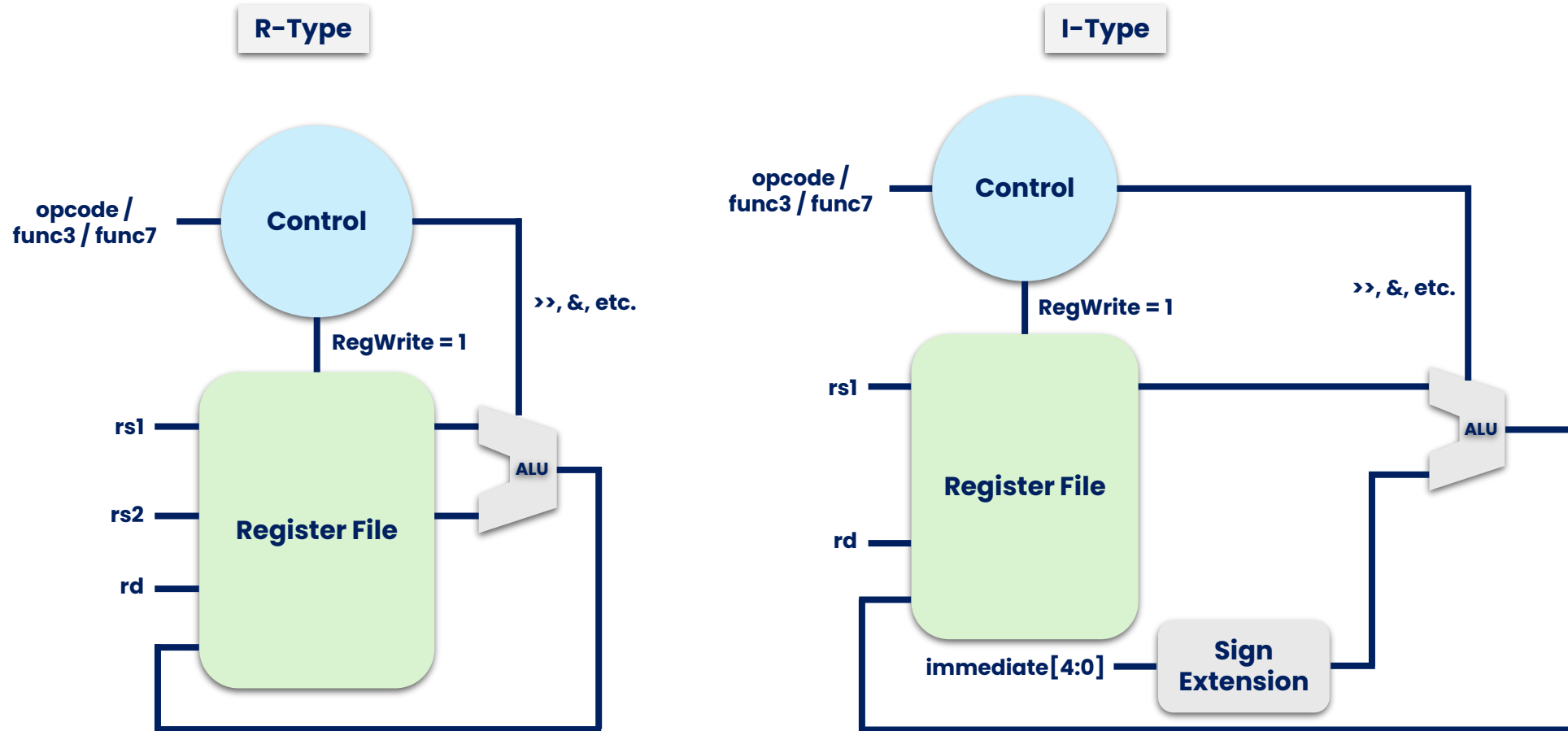
- R-type or I-type Encoding Format

Type		Field				
(field size)	7 bits	5 bits	5 bits	3 bits	5 bits	7 bits
R-type	func7	rs2	rs1	func3	rd	opcode
I-type	func7/func6	immediate	rs1	func3	rd	opcode

- Immediate size in **slli**, **srli** and **srai** Instructions
 - What is the value of immediate needed for these instructions?
 - At max?
 - Can be 31 for 32-bit registers in 32-bit ISA!
 - Can be 63 for 64-bit registers in 64-bit ISA!
 - Because shifting by more than these amounts will be meaningless!
 - 31 can be encoded in 5 bits!
 - 63 can be encoded in 6 bits!
 - So, we do not need to have 12-bit immediate!
 - Only 5-bit / 6-bit immediate is enough!
 - Remaining 7 bits of the immediate are called func7 in 32-bit architecture
 - Format is {func7, immediate, rs1, funct3, rd, opcode}
 - In case of 64-bit architecture, remaining 6 bits are called as func6
 - Format is {func6, immediate, rs1, funct3, rd, opcode}

Logical and Shift Instructions – Microarchitecture

- Implementation is similar to arithmetic instructions



Load Instruction – Machine Language

- I-type Encoding Format
 - Immediate field is the offset address!

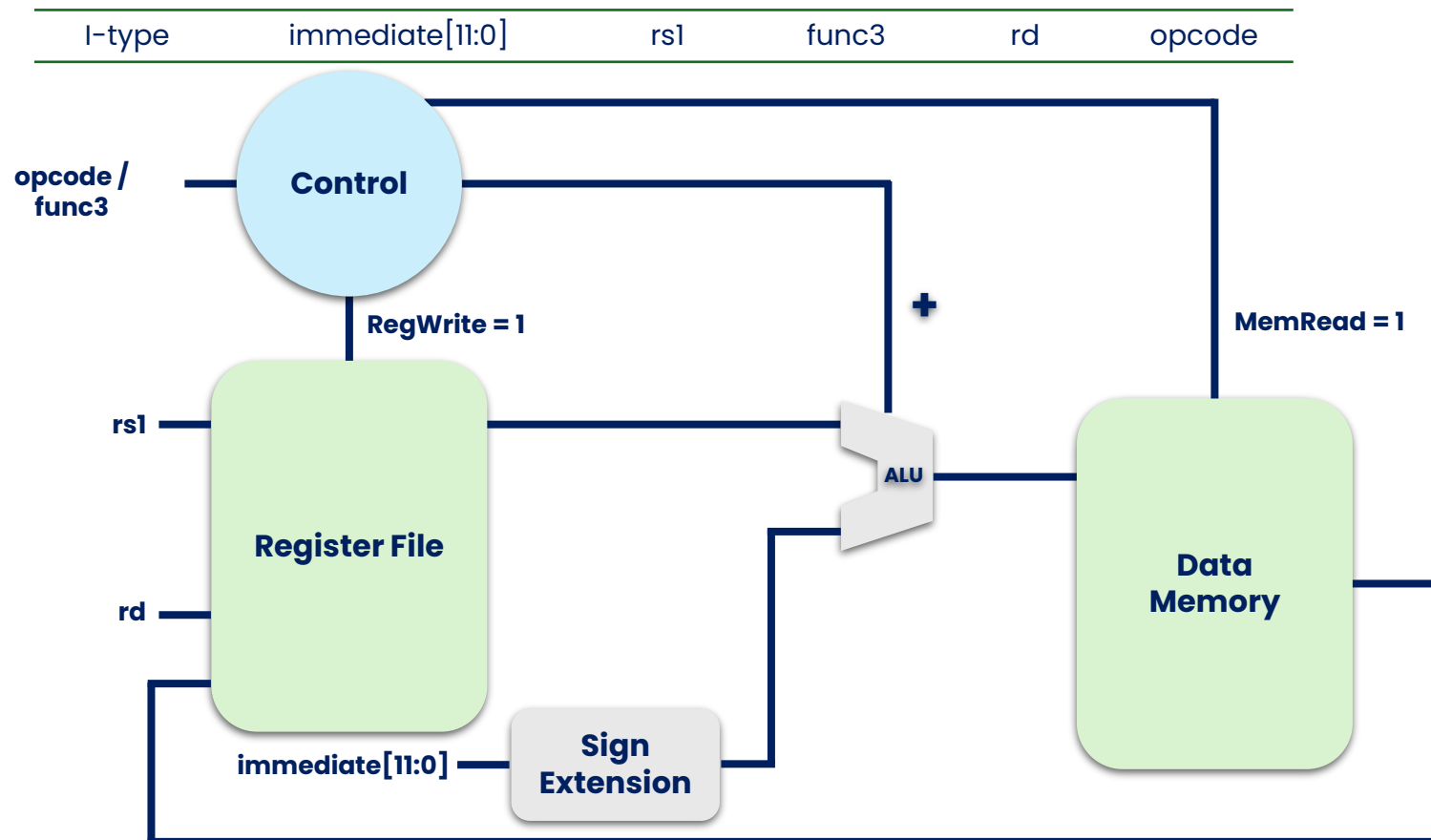
Type	Field						Operations
(field size)	7 bits	5 bits	5 bits	3 bits	5 bits	7 bits	Arithmetic, Logical, Shift and Loads!
I-type	immediate[11:0]		rs1	func3	rd	opcode	

- Example
 - lw x9, 240(x10)

31					0
immediate[11:0]		rs1	funct3	rd	opcode
12 bits		5 bits	3 bits	5 bits	7 bits
240		10	2	9	3
000011110000		01010	010	01001	0000011
0000 1111 0000 0101 0010 0100 1000 0011					
0F052483					

Load Instruction – Microarchitecture

- How a load instruction can be implemented?
 - Example
 - lw x1, 8(x5)



Store Instruction – Machine Language

- Should it be **I-type Encoding Format** just like load?
 - No, think why?
 - Unlike load, store has no destination!
 - Store has two source operands instead of one!
 - One for address just like load!
 - Another containing data to be stored to memory!
 - It uses a unique type (only for stores)!
- S-type Encoding Format

Type		Field					Operations
(field size)	7 bits	5 bits	5 bits	3 bits	5 bits	7 bits	Stores
S-type	immed[11:5]	rs2	rs1	func3	immed[4:0]	opcode	

- Why is immediate divided into two parts?
 - To keep rs2 and rs1 position fixed in different instructions!
 - Simplifies implementation!
 - Registers can be read in parallel with the instruction decoding!

Store Instruction – Machine Language

- S-type Encoding Format

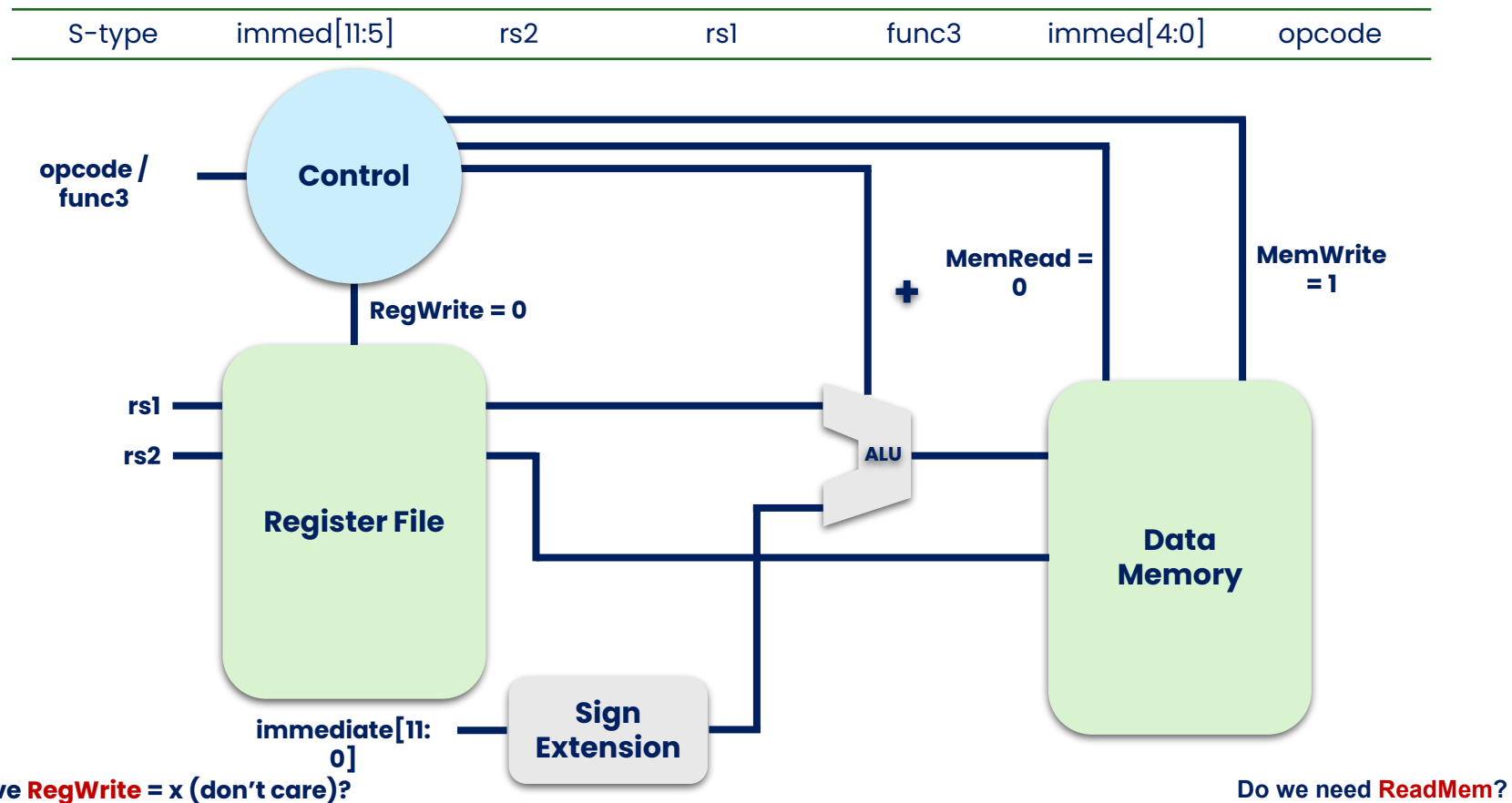
Type		Field					Operations
(field size)	7 bits	5 bits	5 bits	3 bits	5 bits	7 bits	Stores
S-type	immed[11:5]	rs2	rs1	func3	immed[4:0]	opcode	

- Example
 - sw x9, 240(x3)
 - Which is rs1 and which is rs2?
 - rs1 is same as in load (base address register)!
 - 240 = 000011110000 = [7 16] (in two parts)

31						0
immediate[11:5]		rs2	rs1	funct3	immediate[4:0]	opcode
7 bits		5 bits	5 bits	3 bits	5 bits	7 bits
7		9	3	2	16	35
0000111		01001	00011	010	10000	0100011
0000 1110 1001 0001 1010 1000 0010 0011						
0E91A823						

Store Instruction – Microarchitecture

- How a store instruction can be implemented?
 - Example
 - sw x1, 8(x5)



Conditional Branch Instructions – Machine Language

- B-type Encoding Format

Type	Field						Operations
(field size)	7 bits	5 bits	5 bits	3 bits	5 bits	7 bits	Conditional Branches
B-type	immed[12,10:5]	rs2	rs1	func3	immed[4:1,11]	opcode	

- What is the immediate value in `beq x1, x2, target`?
 - PC-Relative addressing
 - Target address is encoded in terms of offset (bytes) from current location!
 - What should be added to current PC to get to the target!!
 - Target Address = Current Address in PC + Immediate Value
 - Immediate is signed value (can be negative / positive)
 - To jump forward as well as backward!
- Why is immediate encoded like this {12,10:5 ... 4:1,11} ?
 - To keep immediate bits (as much as possible) at same locations in different instructions!
 - Less logic required to extract immediate from instruction in different types of instructions!

Conditional Branch Instructions – Machine Language

- B-type Encoding Format

Type	Field						Operations
(field size)	7 bits	5 bits	5 bits	3 bits	5 bits	7 bits	Conditional Branches
B-type	immed[12,10:5]	rs2	rs1	func3	immed[4:1,11]	opcode	

- Wait .. where is bit 0 in the immediate ?
 - Instructions always have even number of bytes!
 - So, target is always even number of bytes away!
 - An LSB of an even number is always 0!
 - We need not encode it in instruction!
 - Instead with 12 bits for immediate we can encode bit 12:1 of immediate!
 - Effectively, we are encoding a 13-bit immediate (with LSB understood to be 0)
 - Range of target?
 - -2^{12} to $+2^{12}-1$ bytes away from current instruction's location!
 - In instruction, we specify how many half-words away the target is!

Conditional Branch Instructions – Machine Language

- B-type Encoding Format

Type		Field					Operations
(field size)	7 bits	5 bits	5 bits	3 bits	5 bits	7 bits	Conditional Branches
B-type	immed[12,10:5]	rs2	rs1	func3	immed[4:1,11]	opcode	

- If every instruction is 32-bit or 4 bytes in size, two LSBs will always be 0, why we only omit 1 LSB while encoding immediate?
 - RISC-V compressed extension allows 16-bit instructions!
 - So, in general, we are only sure of 1 LSB to be 0, not 2!

Conditional Branch Instructions – Machine Language

- B-type Encoding Format

Type		Field					Operations
(field size)	7 bits	5 bits	5 bits	3 bits	5 bits	7 bits	Conditional Branches
B-type	immed[12,10:5]	rs2	rs1	func3	immed[4:1,11]	opcode	

- Example

- 80012: bne x9, x24, **Exit** # instruction is at address 80012 in memory!
- 80016: add x1, x2, x3
- 80020: sub x2, x5, x6
- 80024: **Exit**:
- Immediate in encoding bne x9, x24, Exit?
 - Immediate = 12 (target is 12 (01100) bytes away or 6 (0110) halfwords away)!

Conditional Branch Instructions – Machine Language

- B-type Encoding Format

Type	Field						Operations
(field size)	7 bits	5 bits	5 bits	3 bits	5 bits	7 bits	Conditional Branches
B-type	immed[12,10:5]	rs2	rs1	func3	immed[4:1,11]	opcode	

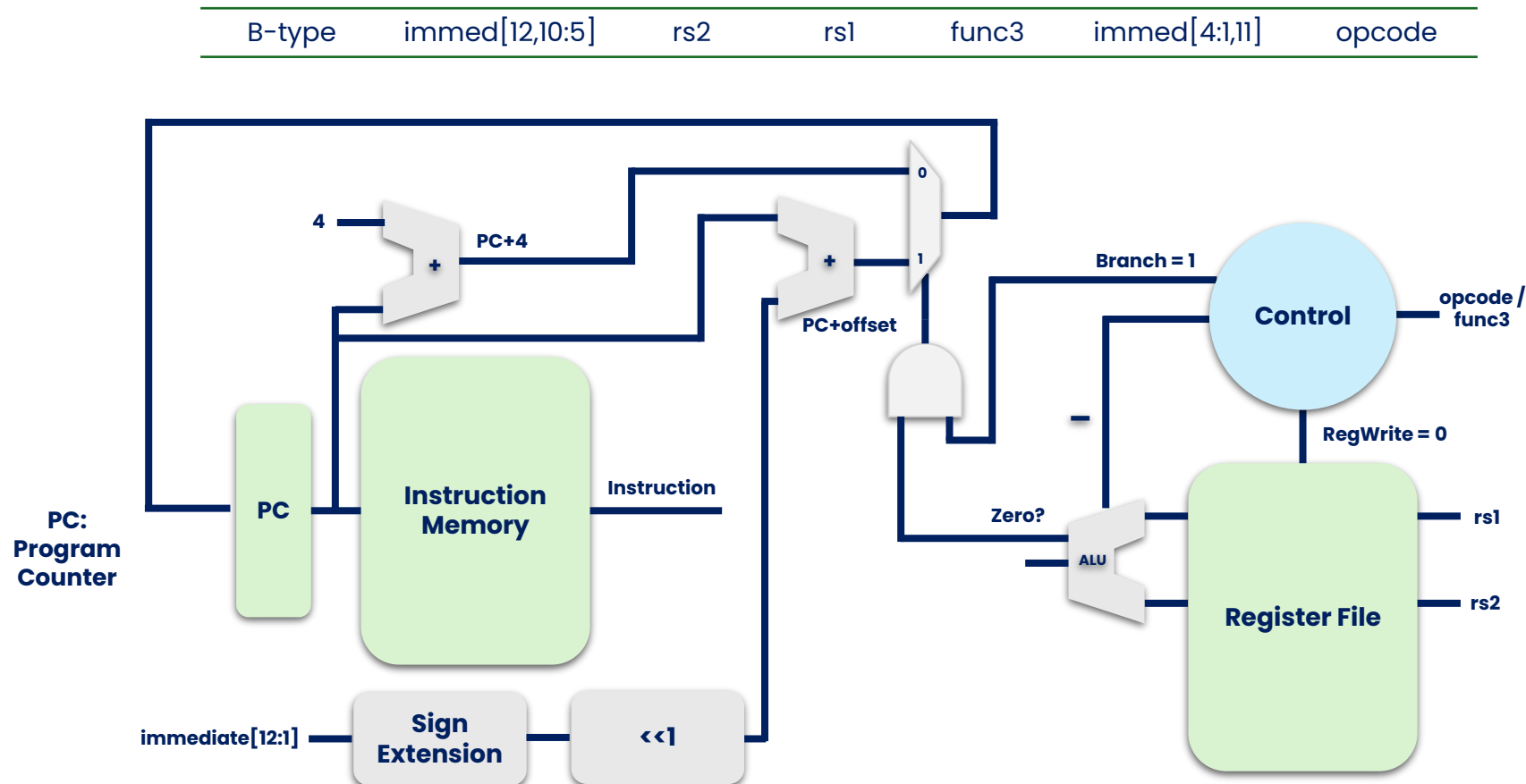
- Example

- 80012: bne x9, x24, **Exit** # instruction is at address 80012 in memory!
- ...
- 80024: **Exit:**

31						0
immediate[12,10:5]		rs2	rs1	funct3	immediate[4:1,11]	opcode
7 bits		5 bits	5 bits	3 bits	5 bits	7 bits
0		24	9	1	12	99
0000000		11000	01001	001	01100	1100011
0000 0001 1000 0100 1001 0110 0110 0011						
01849663						

Conditional Branch Instructions – Microarchitecture

- What needs to be implemented for branches and how?
 - Example
 - beq x5, x6, next



Unconditional Jump Instructions – Machine Language

- J-type Encoding Format

Type		Field					Operations
(field size)	7 bits	5 bits	5 bits	3 bits	5 bits	7 bits	Unconditional jump (jal)
J-type	immediate[20:1,11,19:12]				rd	opcode	

- Example
 - jal x1, repeat
 - rd = 1
 - immediate[20:1] = number of half-words to **repeat** (target address)!
 - immediate[20:0] = number of bytes to **repeat** (target address)!
- What is range of offset (bytes) to target address for **jal**?
 - -2^{20} to $+2^{20}-1$ bytes away from current instruction's location!

Unconditional Jump Instructions – Machine Language

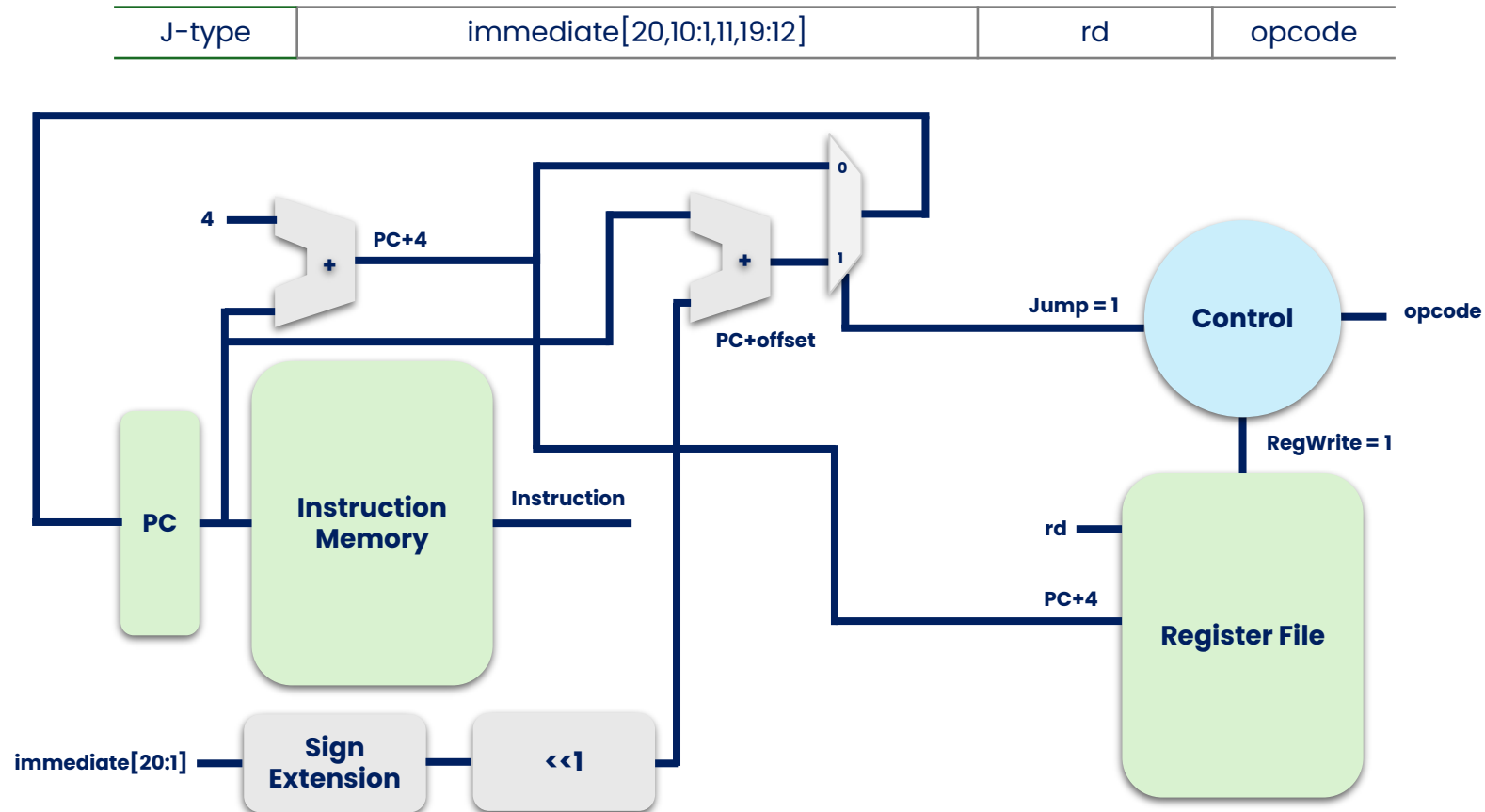
- What about encoding of **jalr** instruction?
 - Example
 - `jalr x0, 0(ra)`
 - Use J-type encoding format?
 - No, it has to use I-type format!

Type		Field					Operations
(field size)	7 bits	5 bits	5 bits	3 bits	5 bits	7 bits	Arithmetic, Logical, Shift, Loads, jalr
I-type	immediate[11:0]		rs1	func3	rd	opcode	

- What is **rs1** and what is **rd** in **jalr**?
 - rs1 = base address register, rd = register to save return address!
- What about range of **jalr** instruction?
 - Uses `{register+offset}` to specify the target address, so address can be any 32-bit / 64-bit address!

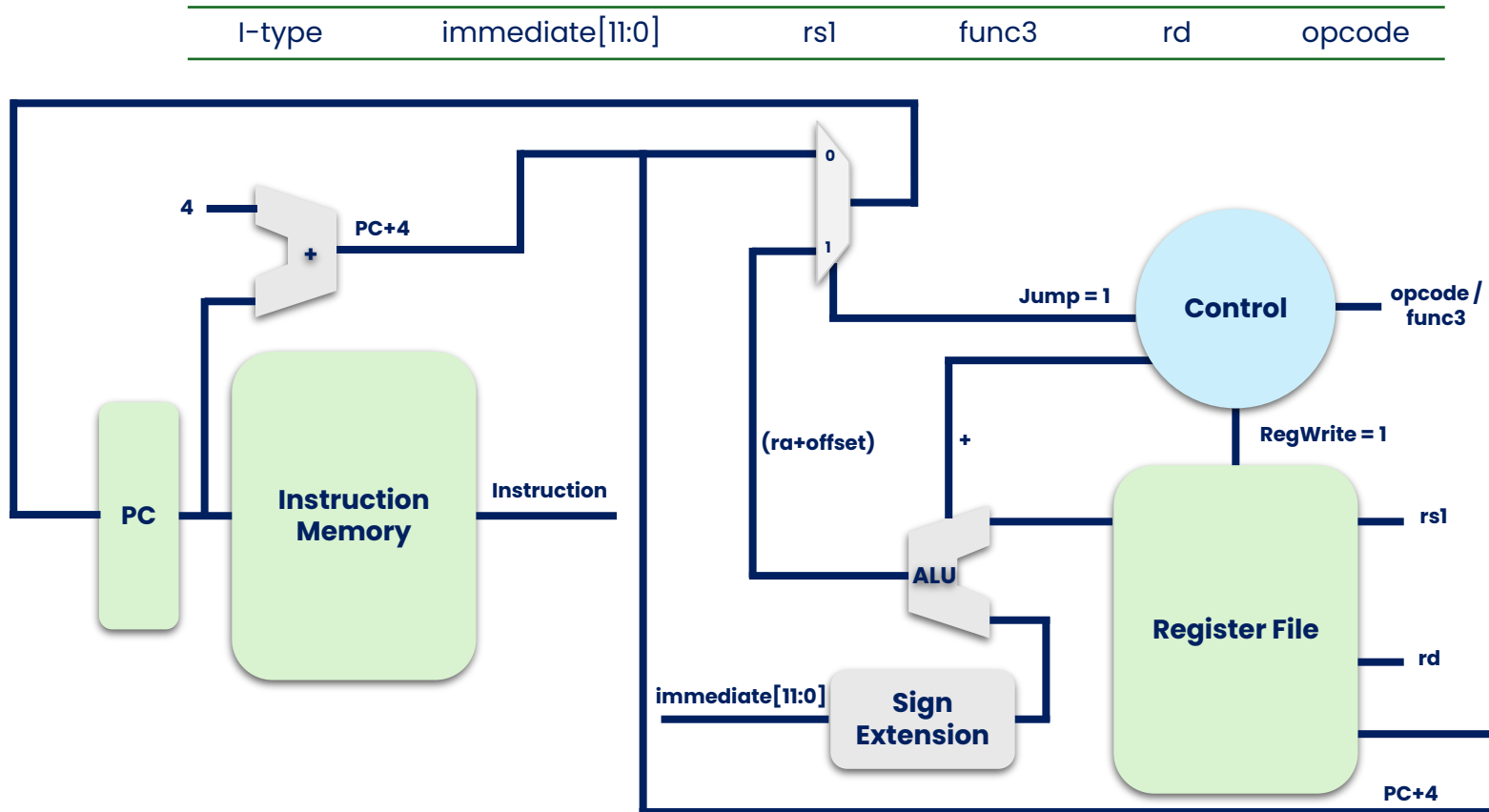
Unconditional Jump Instructions – Microarchitecture

- Implementing **jal** instruction ...
 - Example
 - jal ra, printf**



Unconditional Jump Instructions – Microarchitecture

- Implementing **jalr** instruction ...
 - Example
 - jalr ra, 0(ra)**



LUI and AUIPC Instructions – Machine Language

- U-Type Encoding Format

Type		Field					Operations
(field size)	7 bits	5 bits	5 bits	3 bits	5 bits	7 bits	lui and auipc instructions
U-type	immediate[31:12]				rd	opcode	

- Note
 - immediate[31:12]** doesn't mean that upper 20 bits of immediate value (as specified in instruction) are used
 - it means** 20-bit immediate is placed in upper 20 bits (31:12) of register rd!
- Example
 - lui x19, **976**
 - immediate[31:12] = **976 = 00000000 00111101 0000**
 - x19 = **00000000 00111101 00000000 00000000**

LUI and AUIPC Instructions – Microarchitecture

- How to implement lui and AUIPC?
 - **LUI**
 - Take 20-bit immediate from instruction, append 12-bit 0's towards right and write to rd!
 - **AUIPC**
 - Take 20-bit immediate from instruction, append 12-bit 0's towards right, add it to PC and write to rd!

Summary of Instruction Formats

R-Type: All Instructions

Instruction	Name	Description	Type	Opcode	Funct3	Funct7
add	ADD	$rd = rs1 + rs2$	R	011 0011	000	0000000
sub	SUBtract	$rd = rs1 - rs2$	R	011 0011	000	0100000
and	bitwise AND	$rd = rs1 \& rs2$	R	011 0011	111	0000000
or	bitwise OR	$rd = rs1 rs2$	R	011 0011	110	0000000
xor	bitwise XOR	$rd = rs1 \wedge rs2$	R	011 0011	100	0000000
sll	Shift Left Logical	$rd = rs1 \ll rs2$	R	011 0011	001	0000000
srl	Shift Right Logical	$rd = rs1 \gg rs2$ (Zero-extend)	R	011 0011	101	0000000
sra	Shift Right Arithmetic	$rd = rs1 \gg rs2$ (Sign-extend)	R	011 0011	101	0100000
slt	Set Less Than (signed)	$rd = (rs1 < rs2) ? 1 : 0$	R	011 0011	010	0000000
sltu	Set Less Than (Unsigned)	$rd = (rs1 < rs2) ? 1 : 0$	R	011 0011	011	0000000

I-Type: Arithmetic Instructions

Instruction	Name	Description	Type	Opcode	Funct3	Funct7
addi	ADD Immediate	$rd = rs1 + imm$	I	001 0011	000	
andi	bitwise AND Immediate	$rd = rs1 \& imm$	I	001 0011	111	
ori	bitwise OR Immediate	$rd = rs1 imm$	I	001 0011	110	
xori	bitwise XOR Immediate	$rd = rs1 \wedge imm$	I	001 0011	100	
slli	Shift Left Logical Immediate	$rd = rs1 \ll imm$	I*	001 0011	001	0000000
srli	Shift Right Logical Immediate	$rd = rs1 \gg imm$ (Zero-extend)	I*	001 0011	101	0000000
srai	Shift Right Arithmetic Immediate	$rd = rs1 \gg imm$ (Sign-extend)	I*	001 0011	101	0100000
slti	Set Less Than Immediate (signed)	$rd = (rs1 < imm) ? 1 : 0$	I	001 0011	010	
sltiu	Set Less Than Immediate (Unsigned)	$rd = (rs1 < imm) ? 1 : 0$	I	001 0011	011	

I-Type: Load and Jump Instructions

Instruction	Name	Description	Type	Opcode	Funct3	Funct7
lb rd imm(rs1)	Load Byte	rd = 1 byte of memory at address rs1 + imm, sign-extended	I	000 0011	000	
lbu rd imm(rs1)	Load Byte (Unsigned)	rd = 1 byte of memory at address rs1 + imm, zero-extended	I	000 0011	100	
lh rd imm(rs1)	Load Half-word	rd = 2 bytes of memory starting at address rs1 + imm, sign-extended	I	000 0011	001	
lhu rd imm(rs1)	Load Half-word (Unsigned)	rd = 2 bytes of memory starting at address rs1 + imm, zero-extended	I	000 0011	101	
lw rd imm(rs1)	Load Word	rd = 4 bytes of memory starting at address rs1 + imm	I	000 0011	010	
jalu rd rs1 imm	Jump And Link Register	rd = PC + 4; PC = rs1 + imm	I	110 0111	000	

S-Type: All Instructions

Instruction	Name	Description	Type	Opcode	Funct3	Funct7
sb rs2 imm(rs1)	Store Byte	Stores least-significant byte of rs2 at the address rs1 + imm in memory	S	010 0011	000	
sh rs2 imm(rs1)	Store Half-word	Stores the 2 least-significant bytes of rs2 starting at the address rs1 + imm in memory	S	010 0011	001	
sw rs2 imm(rs1)	Store Word	Stores rs2 starting at the address rs1 + imm in memory	S	010 0011	010	

- Warning: **rs2** comes before **rs1** in store instructions!
 - This is because we want rs1 to always be the register that gets added to immediates, to simplify our circuitry

U-Type: All Instructions

Instruction	Name	Description	Type	Opcode	Funct3
auipc rd imm	Add Upper Immediate to PC	$rd = PC + (imm \ll 12)$	U	001 0111	
lui rd imm	Load Upper Immediate	$rd = imm \ll 12$	U	011 0111	

B-Type: All Instructions

Instruction	Name	Description	Type	Opcode	Funct3
beq rs1 rs2 label	Branch if Equal	if (rs1 == rs2) PC = PC + offset	B	110 0011	000
bge rs1 rs2 label	Branch if Greater or Equal (signed)	if (rs1 >= rs2) PC = PC + offset	B	110 0011	101
bgeu rs1 rs2 label	Branch if Greater or Equal (Unsigned)	if (rs1 >= rs2) PC = PC + offset	B	110 0011	111
blt rs1 rs2 label	Branch if Less Than (signed)	if (rs1 < rs2) PC = PC + offset	B	110 0011	100
bltu rs1 rs2 label	Branch if Less Than (Unsigned)	if (rs1 < rs2) PC = PC + offset	B	110 0011	110
bne rs1 rs2 label	Branch if Not Equal	if (rs1 != rs2) PC = PC + offset	B	110 0011	001

J-Type: All Instructions

Instruction	Name	Description	Type	Opcode	Funct3
jal rd label	Jump and link	rd = PC + 4 PC = PC + offset	J	110 1111	

How to handle immediates larger than you can store

- R-type and U-type instructions
 - Unneeded, since they either don't have immediates or have very specific use cases that never need to exceed the given immediate length
- I-type and S-type instructions
 - For arithmetic instructions, it's generally possible to store the immediate in a temporary first
 - Ex. if we want to do "xori t0 t1 0xDEADBEEF", we can do:
li t2 0xDEADBEEF
xor t0 t1 t2
 - For loads and stores, we can add the offset first, then do a 0-offset load (as with variable offset loads)

How to handle immediates larger than you can store

- B-type and J-type instructions
 - If a branch is:
 - Within 1024 instructions?
 - Branch normally (ex. beq t0 t1 Label)
 - Greater than 1024 instructions?
 - Invert the branch condition, and do a j instruction instead:
bne t0 t1 **Next**
j **Label**
Next:
 - If a jump is:
 - Within 218 instructions?
 - Jump normally (ex. j Label)
 - Greater than 218 instructions?
 - Do an auipc, then use jalr's immediate to offset the rest:
auipc t0 0x12345
jalr ra t0 0x678

Thank You

